

Evaluation Modulaire

Application mobile - Goals

Contexte

Une jeune entreprise spécialisée dans le sport et le bien-être souhaite lancer une application mobile appelée *Goals*.

L'application doit permettre à ses utilisateurs de consulter rapidement leurs statistiques sportives et de garder en tête les objectifs qu'ils se sont fixés.

L'entreprise dispose déjà d'une API capable de fournir les statistiques et les objectifs d'un utilisateur. Elle souhaite désormais disposer d'une première version de l'application mobile Android.

Votre mission

Vous devez développer l'interface mobile de *Goals* à partir des maquettes fournies.

L'application comportera deux écrans accessibles depuis une barre de navigation située en bas de l'écran :

- un écran *Accueil* affichant les données sportives provenant de l'API ;
- un écran *Profil* affichant des informations utilisateur fictives.

Aucun développement de serveur ou de base de données n'est demandé.

Écran *Accueil*

L'écran d'accueil constitue le tableau de bord principal de l'application.

En-tête

La partie supérieure de l'écran doit contenir :

- le nom de l'application *Goals* ;
- un court message motivant, comme *Continuez vos efforts !*

Statistiques sportives

L'écran doit afficher trois statistiques provenant de l'API :

Statistique	Description
Séances	Nombre total de séances réalisées
Temps	Temps total consacré au sport

Statistique	Description
Calories	Nombre total de calories dépensées

Chaque statistique doit être présentée dans une carte distincte.

Pour la valeur du temps, les minutes pourront être converties en heures et minutes. Par exemple, 210 minutes seront affichées sous la forme 3 h 30.

Objectifs

Sous les statistiques, l'application doit afficher la liste des objectifs sportifs de l'utilisateur.

Chaque objectif doit présenter :

- son titre ;
- la cible à atteindre.

Exemple :

Courir plus souvent
3 séances/semaine

La liste doit être construite dynamiquement à partir des données retournées par l'API. L'application doit donc continuer à fonctionner si le nombre d'objectifs évolue.

Écran *Profil*

L'écran *Profil* doit être accessible depuis la barre de navigation inférieure.

Les informations affichées sur cet écran **ne proviennent pas de l'API**. Elles seront directement intégrées à l'application sous la forme de données fictives.

Les données à afficher sont les suivantes :

```
JSON
{
  "name": "Léo Martin",
  "description": "Sportif amateur",
  "mainGoal": "Courir 3 fois par semaine",
  "targetWeight": 75,
  "favoriteActivity": "Running",
  "level": "Débutant"
}
```

L'écran doit présenter :

- le nom de l'utilisateur ;

- une courte description ;
- son objectif principal ;
- son objectif de poids ;
- son activité favorite ;
- son niveau sportif.

Une image ou une illustration de profil peut être utilisée, mais elle n'est pas obligatoire.

Le bouton *Modifier* visible sur la maquette peut être uniquement visuel. Il n'est pas nécessaire de développer un formulaire de modification.

Navigation

Une barre de navigation doit être visible en bas de l'application.

Elle contient deux entrées :

- Accueil ;
- Profil.

L'entrée correspondant à l'écran actuellement affiché doit être visuellement identifiable.

L'utilisateur doit pouvoir passer d'un écran à l'autre en appuyant sur ces entrées.

API disponible

Les données du tableau de bord doivent être récupérées avec la requête suivante :

None

```
GET https://labs.nicolas.sh/reactnative/evmod/api/
```

Note : les résultats de cet appel API changent à chaque chargement. Pour les figer, utiliser un paramètre *fixed=true* à passer en query string.

Exemple de réponse

JSON

```
{
  "stats": {
    "sessions": 34,
    "spentTime": 210,
    "kcal": 753
  },
  "goals": [
    {
      "title": "Courir plus souvent",
```

```
    "target": "3 séances/semaine"
  },
  {
    "title": "Perdre du poids",
    "target": "-3 kg"
  }
]
```

Gestion du chargement

Pendant la récupération des données, l'utilisateur doit être informé que le chargement est en cours.

Vous pouvez afficher :

- un indicateur de chargement ;
- un message comme *Chargement de votre activité...*

Les statistiques et les objectifs ne doivent pas être affichés tant que les données ne sont pas disponibles.

Gestion des erreurs

Lorsque l'API ne peut pas être contactée, l'application doit afficher un message compréhensible.

Exemple :

Impossible de récupérer vos données sportives.
Vérifiez votre connexion et réessayez.

Un bouton *Réessayer* doit permettre de relancer la requête.

L'application ne doit pas afficher de message d'erreur technique destiné aux développeurs.

Maquettes

Les maquettes fournies présentent :

- l'écran d'accueil avec les statistiques et les objectifs ;
- l'écran de profil ;
- la barre de navigation.

Vous devez respecter la structure générale, les espacements, la hiérarchie des informations et l'esprit graphique des maquettes.

Contraintes

- L'application doit fonctionner sur Android.
- Les statistiques et les objectifs doivent provenir de l'API.
- Les objectifs ne doivent pas être écrits directement dans l'interface.
- Les données du profil peuvent être écrites directement dans l'application.
- L'application doit gérer les états de chargement, de succès et d'erreur.
- La navigation entre *Accueil* et *Profil* doit être fonctionnelle.
- Les styles doivent être regroupés et organisés proprement.
- L'interface doit rester lisible sur différentes tailles d'écran.
- Aucun système d'authentification n'est demandé.
- Il n'est pas demandé de créer, modifier ou supprimer des objectifs.
- Il n'est pas demandé de sauvegarder les données localement.

Livrables

À la fin de l'évaluation, vous devrez fournir :

1. le projet React Native complet ;
2. une application fonctionnelle sur Android ;
3. une démonstration sur un émulateur ou un appareil Android ([capture vidéo](#)) ;

Barème

Critère évalué	Points
Structure et respect des patterns ReactNative	6
Récupération des données depuis l'API	2
Affichage des statistiques	2
Affichage dynamique des objectifs	1
Gestion du chargement	0,5
Gestion des erreurs et nouvelle tentative	1
Navigation entre les deux écrans	2,5
Réalisation de l'écran <i>Profil</i>	2
Qualité visuelle et respect des maquettes	2
Organisation et lisibilité du code	1
Total	20

Bonus

Jusqu'à 2 points bonus pourront être accordés pour :

- une fonction de *pull to refresh* pour rafraîchir les données de la page d'accueil ;
- une interface particulièrement soignée ;
- une bonne adaptation aux différentes tailles d'écran ;
- l'ajout d'éléments améliorant l'accessibilité.

La note finale reste plafonnée à 20.

Rendu

Vous devez rendre votre programme sous la forme d'un repository Github (public ou partagé avec moi). Le lien sera à déposer dans le Google Classroom.

Attention, veillez à bien inclure à votre repository les éléments qui me permettront de recréer votre environnement de développement, sans toutefois inclure les éléments qui sont généralement ignorés dans un projet réel (node_modules...).

Avant de valider, **vérifiez bien** votre rendu. Si les fichiers sont introuvables, que le repo n'est pas à jour ou pas accessible, le devoir sera considéré comme non rendu.

Utilisation de l'IA

Cette évaluation a pour objectif de vérifier vos compétences et votre compréhension des notions abordées en session.

Si, lors de la correction, votre rendu semble avoir été réalisé avec l'aide d'une IA ou présente un écart important avec votre niveau habituel et les contenus étudiés, votre note pourra être temporairement mise en attente. Vous serez alors invité à répondre à quelques questions à l'oral afin d'expliquer votre démarche et certains éléments de votre travail.

Si cet échange confirme que vous maîtrisez ce que vous avez produit, la note sera validée normalement. En revanche, si vous n'êtes pas en mesure d'expliquer votre rendu ou si un décalage important est constaté, la note pourra être ramenée à 0.

Annexe - Maquettes

Vous pouvez utiliser cette librairie pour les icônes : [oblador/react-native-vector-icons](https://oblador.github.io/react-native-vector-icons)



Couleur	Variante Light	Variante Dark
Primaire (vert)	 #EEF9F3	 #119B4A
Orange	 #FEEDE0	 #FD660B
Bleu	 #E9EEFC	 #166CFD